



ENGINEERING DOCUMENTATION • WRITTEN FOR A FIRST-TIME READER

JanMitra AI Project Overview

A ground-up walkthrough of what this codebase is, how its pieces fit together, and what every file in it does — for someone opening the repository for the first time.

Branch `feature/react-auth-rebuild` **Stack** `FastAPI` • `SQLite` • `React` + `TypeScript` • `Sarvam AI`

Prepared `2026-08-06`

What JanMitra AI actually is

A civic complaint app for garbage collection — but built around one specific problem: the citizen reporting the problem and the municipal worker fixing it often don't speak the same language.

In most Indian towns and cities, a citizen might be most comfortable speaking Marathi or Hindi, while the sanitation worker or ward office might work primarily in a different language, or vice-versa. Ordinary complaint apps assume everyone involved shares one language. JanMitra AI doesn't: a citizen speaks or types a complaint in *their* language, and the app automatically translates it so a worker can read it in *theirs* — with no manual translation step for either person.

Under the hood, it's a fairly conventional web app — a database, a backend API, a browser-based frontend — with one added ingredient: every complaint passes through a small pipeline of AI calls (speech-to-text, spelling cleanup, translation, summarization) provided by an Indian AI company called **Sarvam AI**, who specialize in Indian-language models.

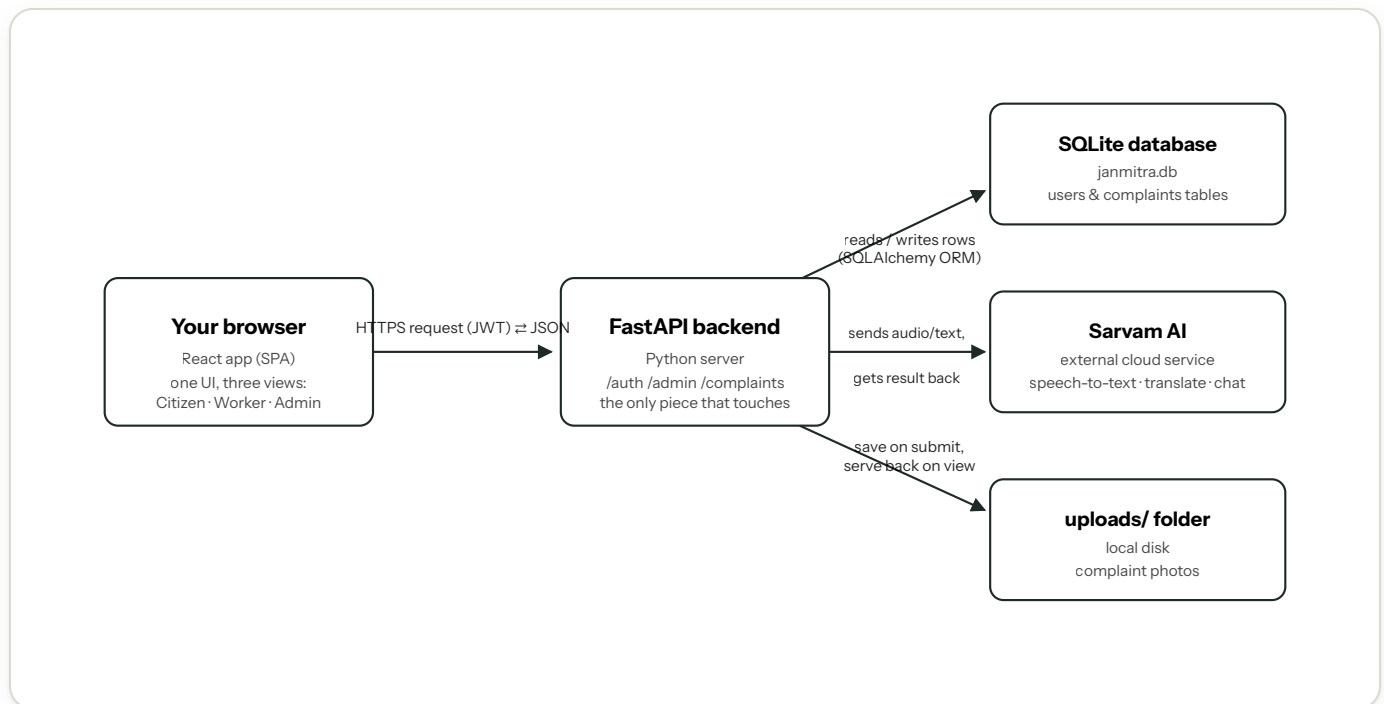
The three kinds of people who use it

ROLE	WHO THEY ARE	WHAT THEY CAN DO	HOW THEIR ACCOUNT IS CREATED
CITIZEN	A resident reporting a problem	Submit complaints (voice or text), attach a photo, see their own complaints and their status	Signs up themselves, anytime, on the app
WORKER	Sanitation staff for one specific ward	See every complaint filed for their ward, mark complaints resolved	Created only by a Super Admin — cannot self sign-up
SUPER ADMIN	Municipal office overseeing all wards	See every complaint everywhere, create new worker accounts, see workload per worker	Seeded directly into the database via a command-line script — never created through the app itself

That last row is deliberate, not an oversight: there is no button, form, or API endpoint anywhere in this codebase that can create a Super Admin. The very first admin account for a real deployment is planted straight into the database by whoever is setting the system up, using `scripts/seed_admin.py`. From then on, that admin can create worker accounts through the app.

System architecture

Five moving pieces, only one of which — Sarvam AI — lives outside this codebase. Everything else runs on your own machine (or wherever it's deployed).



What this shows: your browser talks to exactly one thing — the FastAPI backend — over HTTPS, proving who you are with a signed token (JWT) on every request. The backend is the only piece that talks to the database, to Sarvam AI, and to the folder of saved photos; the browser never reaches any of those directly.

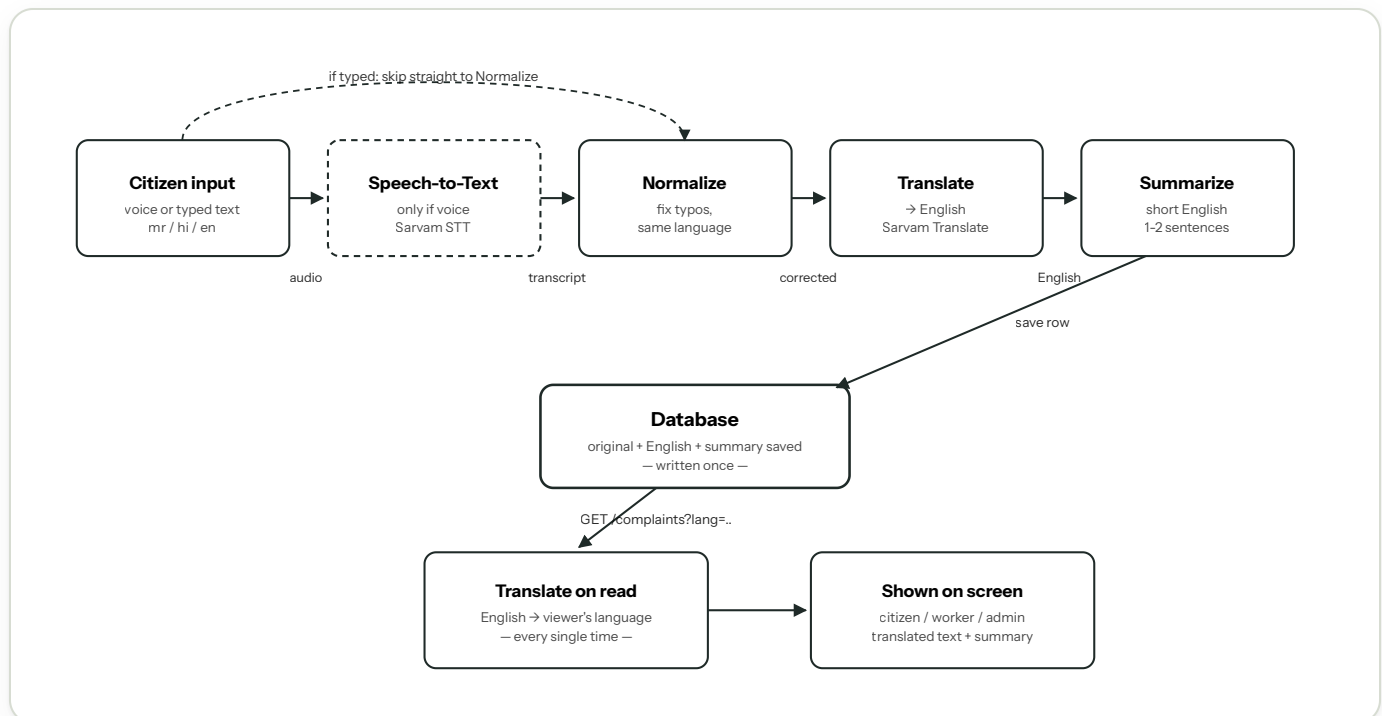
Why it's shaped this way

This is a classic three-tier shape: a **frontend** (what you see and click), a **backend** (the rules and logic, and the only thing allowed to touch the database), and **storage** (the database and the photo folder). The browser is never trusted with direct database access or an AI API key — it only ever talks to the backend, and the backend decides what it's allowed to see based on who it's logged in as.

Sarvam AI sits outside the box marked "this codebase" because it's a separate company's service, reached over the internet with an API key — same idea as how a weather app calls a weather service instead of running its own weather satellites.

How one complaint actually flows through the system

This is the part that makes the app's whole premise work. Two things happen at very different times: **submitting** a complaint (happens once) and **viewing** a complaint (happens every time anyone opens a complaint list) — and they're not symmetric.



What this shows: a complaint is translated to English exactly *once*, when it's submitted, and that English text is what's permanently saved. But it is translated *again, fresh, every single time* someone views the complaint list in a non-English language — the database never stores a Hindi or Marathi copy. --- marks the one step (speech-to-text) that's skipped entirely when the citizen typed instead of spoke.

Why translate on every read, instead of once and storing all three languages?

Simplicity, at a real cost. Storing one canonical English copy per complaint means the database schema never has to change if a fourth or fifth language is added later — there's no "Marathi column" and "Hindi column" to maintain. The trade-off is that every time a worker opens their queue, the backend makes a fresh translation call for every complaint on screen, which is slower than reading an already-translated row would be. For a small pilot this is a fine trade; it's the kind of decision worth revisiting if the complaint volume or the number of viewers grows a lot.

The four AI steps, in plain terms

STEP	WHAT IT DOES	SARVAM MODEL USED	WHEN IT RUNS
Speech-to-text	Turns a voice recording into text, in whatever language the citizen spoke	saaras:v3	Only if the citizen recorded audio instead of typing
Normalize	Fixes obvious spelling/typing slips, without translating or changing meaning, in the citizen's own language	sarvam-105b (chat)	Always, right after the text exists, before translation
Translate	Converts the (now cleaned-up) text into English for storage, and later converts the stored English back into whichever language a viewer wants to read it in	sarvam-translate:v1	Once at submit time (→ English); again on every read (→ viewer's language)
Summarize	Writes a short 1-2 sentence English summary, so a worker skimming a long queue doesn't have to read every full complaint	sarvam-105b (chat)	Once at submit time

WORTH KNOWING

Every one of these AI calls can fail — a network blip, or the AI service being slow or unavailable. Normalize and Summarize are both written to **degrade gracefully**: if Normalize fails, the complaint is stored with its original, un-cleaned-up text instead of blocking submission entirely. The design intent is that a citizen's report should never be lost just because an AI nicety didn't complete — though at the time of writing, the summary step on this branch still raises an error rather than falling back (see §10).

Words used throughout this document

If any of this is already familiar, skip ahead — everything below assumes you know none of it.

API

A defined set of URLs a program can call to ask another program to do something — here, the browser calls the backend's API to fetch or submit complaints.

Backend

The server-side program (here, FastAPI/Python) that holds the real logic and is the only thing allowed to touch the database.

SPA

Single-Page Application: the browser loads one HTML page once, and JavaScript swaps the content in and out as you navigate, instead of reloading a new page from the server each click.

Hook

A React function starting with `use` (e.g. `useState`, `useAuth`) that lets a component hold state or reuse logic.

JWT

JSON Web Token — a signed, tamper-proof string handed to a user after login, sent back on every later request as proof of who they are. Explained fully in §6.

LLM

Large Language Model — the kind of AI model behind the "Normalize" and "Summarize" steps

Endpoint

One specific URL + method in an API, e.g. `POST /complaints`. Each endpoint does one job.

Frontend

The program that runs in the user's browser (here, a React app) — what you actually see and click.

Component

A reusable, self-contained piece of a React UI — a button, a form, a whole page. Building blocks you assemble into screens.

ORM

Object-Relational Mapper — lets Python code describe database rows as regular objects/classes (see `backend/models.py`) instead of writing raw SQL by hand. This project uses SQLAlchemy.

Environment variable

A configuration value (an API key, a secret, a URL) kept outside the code, in a `.env` file, so secrets are never hardcoded or committed to git.

BCP-47 code

The standard format for language codes with a region, e.g. `mr-IN` for Marathi (India). Sarvam's AI

(Sarvam's `sarvam-105b` chat model).

APIs require this form; the rest of the app uses short codes like `mr`.

The backend (Python / FastAPI)

Everything under `backend/`. Read top to bottom and you're reading the request-handling stack from the outside in: entry point → routes → services → database.

FILE	WHAT IT'S FOR
<code>main.py</code>	The actual FastAPI application. Wires up CORS (which browser origins may call this API), mounts the three route groups, serves uploaded photos as static files at <code>/uploads</code> , and runs <code>init_db()</code> on startup.
<code>config.py</code>	Every setting the app needs — API keys, model names, file-size limits, the JWT secret, supported languages — read once from environment variables (via <code>.env</code>) into one <code>settings</code> object. Nothing else in the codebase reads <code>os.environ</code> directly.
<code>database.py</code>	Creates the SQLAlchemy engine and session factory, and <code>init_db()</code> , which creates any missing tables on startup. There is no formal migration system — this is why: schema changes just mean the app auto-creates whatever tables don't exist yet.
<code>models.py</code>	The two database tables, described as Python classes: <code>Complaint</code> and <code>User</code> . See §7 for the full schema.
<code>deps.py</code>	Two small but load-bearing functions: <code>get_current_user</code> (reads the JWT from a request and resolves it to a real user) and <code>require_role(...)</code> (builds a check that only lets specific roles through). Almost every route depends on one of these.
<code>routes/auth.py</code>	Sign-up, login, "who am I", and profile updates. Sign-up always creates a <code>citizen</code> — this file has no code path that can create anything else.
<code>routes/admin.py</code>	Super-admin-only: create a worker account, list all workers with their open/resolved complaint counts. Every route here is gated by <code>require_role("admin")</code> .
<code>routes/complaints.py</code>	The heart of the app: create a complaint, list complaints (scoped by the caller's role), update a complaint's status, and list which wards have a worker assigned (for the ward picker dropdown).
<code>services/sarvam_client.py</code>	The one place that calls Sarvam's speech-to-text and translation APIs directly. Defines <code>AIServiceError</code> , the exception every AI failure eventually turns into.
<code>services/translation_service.py</code>	A thin, readable wrapper around <code>sarvam_client</code> 's <code>translate()</code> : <code>to_english()</code> and <code>to_language()</code> , so the rest of the code never has to think in BCP-47 codes directly.

FILE	WHAT IT'S FOR
<code>services/normalization_service.py</code>	Spelling cleanup, run in the citizen's own language before translation. Never raises — on any failure it just returns the original text unchanged, since this is a quality nicety, not a required step.
<code>services/summary_service.py</code>	Generates the short English summary via a chat-completion call. On this branch, an empty/failed AI response still raises <code>AIServiceError</code> , which currently fails the whole submission (see §10).
<code>services/complaint_agent.py</code>	Orchestrates the whole pipeline for one complaint: transcribe (if voice) → normalize → translate → summarize → save. This is the class both the route and the tests call into; it doesn't know or care whether the caller is an HTTP request or a test.
<code>prompts/*.txt</code>	The literal text sent to the AI models as instructions — kept as plain files, not hardcoded strings, so a prompt can be tweaked without touching Python code. <code>summary_prompt.txt</code> , <code>normalize_prompt.txt</code> , <code>normalize_system_prompt.txt</code> , <code>system_prompt.txt</code> .
<code>scripts/seed_admin.py</code>	Command-line script: plants the very first Super Admin account directly into the database. Safe to re-run — it checks for an existing account with that phone number first.

The frontend (React + TypeScript)

Everything under `frontend-react/src/`. React apps are built from small, focused files — this one splits cleanly into **pages** (one per screen), **components** (reusable pieces used across pages), and **lib** (logic that isn't a visual thing at all: talking to the API, remembering who's logged in, etc).

Entry point & routing

FILE	WHAT IT'S FOR
<code>main.tsx</code>	The actual entry point — the one file that calls <code>createRoot(...).render(...)</code> to boot React into the page. Wraps the whole app in four "providers" (theme, router, UI language, auth) so every page below can use them.
<code>App.tsx</code>	The route table: which URL path shows which page, and which pages require which role logged in (via <code>ProtectedRoute</code>).

Pages — one per screen

FILE	ROUTE	WHO SEES IT	WHAT IT DOES
LanguageGate.tsx	/	ANYONE	The very first screen: pick Marathi / Hindi / English for the app's own UI text (not the complaint's language — that's chosen separately, per complaint).
Landing.tsx	/welcome	ANYONE	Marketing-style landing page with a headline, and buttons to log in or sign up.
Login.tsx	/login	ANYONE	Phone + password form. On success, redirects to whichever dashboard matches the logged-in user's role.
Signup.tsx	/signup	ANYONE	Name + phone + password form — always creates a citizen account, and explicitly tells sanitation workers to ask their admin for a login instead of signing up here.
CitizenDashboard.tsx	/citizen	CITIZEN	The complaint form (type or speak, pick a language, pick a ward, attach a photo) plus a list of the citizen's own past complaints and their status.
WorkerDashboard.tsx	/worker	WORKER	The queue of complaints for the worker's own ward, filterable by open/resolved, with a "Mark Resolved" button on each.
AdminDashboard.tsx	/admin	ADMIN	Every worker across every ward, with their open/resolved counts, and a button to add a new worker.

Components — reusable pieces

FILE	WHAT IT'S FOR
<code>ProtectedRoute.tsx</code>	Wraps a page and refuses to render it unless someone is logged in <i>and</i> holds one of the allowed roles — otherwise redirects to login, or to that user's own correct dashboard.
<code>TopBar.tsx</code>	The header bar shown on every logged-in page: app name, the current user's name/role, theme toggle, and a settings button.
<code>AddWorkerModal.tsx</code>	The popup form an admin fills in to create a new worker account (name, phone, temporary password, ward, language).
<code>SettingsModal.tsx</code>	Popup for editing your own name and preferred language, or logging out.
<code>ThemeToggle.tsx</code>	The icon button that cycles System → Light → Dark and remembers the choice.

lib/ — logic that isn't a screen

FILE	WHAT IT'S FOR
<code>api.ts</code>	Every single call the frontend ever makes to the backend, in one file. Wraps <code>fetch</code> , attaches the auth token, and turns backend error responses into a typed <code>ApiError</code> .
<code>auth.tsx</code>	Remembers who's logged in (in <code>localStorage</code> , so a page refresh doesn't log you out), and exposes <code>useAuth()</code> to any component that needs to know.
<code>i18n.ts</code>	The app's own UI text (buttons, labels) in all three languages — separate from the AI translation pipeline, which only ever translates complaint content, never the app's interface.
<code>theme.tsx</code>	Light/dark/system theme state, persisted to <code>localStorage</code> .
<code>uiLang.tsx</code>	Which of the three languages the app's own interface is currently shown in (set once, on the LanguageGate screen).
<code>useAudioRecorder.ts</code>	A custom hook wrapping the browser's microphone recording API (<code>MediaRecorder</code>) — start, stop, get back an audio file, with friendly error messages if the microphone is denied or missing.

Tests, and everything at the project root

Tests

`tests/test_auth_routes.py``tests/test_auth_service.py``tests/test_admin_routes.py``tests/test_complaints_api.py``tests/test_normalization_service.py` , `test_summary_service.py` , `test_translation_servic``tests/test_property_based.py``frontend-react/e2e/*.spec.ts`

Project root

FILE / FOLDER	WHAT IT IS
<code>README.md</code>	The project's own front-door documentation (currently describes an earlier, Streamlit-only version of the architecture — this document you're reading reflects the current React + auth branch instead)
<code>future_work.md</code>	An honest, running list of what's explicitly <i>not</i> built yet, organized by milestone — see §11
<code>requirements.txt</code>	Python package versions the backend needs (FastAPI, SQLAlchemy, the Sarvam SDK, bcrypt, pytest, etc.)
<code>frontend-react/package.json</code>	The React app's dependencies and its <code>npm run</code> scripts (dev server, build, lint, e2e tests)
<code>.env</code> / <code>.env.example</code>	Real secrets (gitignored, never committed) vs. a documented template of what variables exist
<code>janmitra.db</code>	The actual SQLite database file — a single file on disk containing every table and row
<code>uploads/</code>	Where complaint photos are physically saved, served back out at <code>/uploads/<filename></code>
<code>frontend/</code>	Legacy. The original Streamlit citizen/worker apps, superseded by <code>frontend-react/</code> . Kept for reference only — it predates the auth system and no longer works against this backend without it.
<code>docs/ai_pipeline_limits.md</code>	A separate, deep investigation into the real-world limits of the AI pipeline (how long can a voice recording be, how big can a complaint get before summarization gets unreliable) — referenced in §10.

The database, in plain terms

SQLite is a database that's just a single file on disk (`janmitra.db`) — no separate server process to install or run, which is why this app can run with zero database setup. A **table** is like a spreadsheet; each **row** is one record; each **column** is one field every row has. There are exactly two tables.

users table

COLUMN	WHAT IT HOLDS
<code>id</code>	Auto-incrementing number identifying this account
<code>phone</code>	Login identifier — must be unique across every account
<code>password_hash</code>	A bcrypt hash of the password. The real password is <i>never</i> stored anywhere.
<code>full_name</code>	Display name
<code>role</code>	One of <code>citizen</code> , <code>worker</code> , <code>admin</code>
<code>preferred_language</code>	Short code (<code>mr</code> / <code>hi</code> / <code>en</code>) used to translate complaint text for this person by default
<code>ward</code>	Which ward this worker is responsible for — blank for citizens and admins
<code>created_at</code>	When the account was created

complaints table

COLUMN	WHAT IT HOLDS
<code>id</code>	Auto-incrementing complaint number (shown to users as <code>JM-00001</code> etc.)
<code>citizen_id</code>	Which user filed it
<code>original_text</code>	Exactly what the citizen said or typed, untouched — kept as the authentic record
<code>original_language</code>	Which language <code>original_text</code> is in
<code>translated_text</code>	The canonical English version — this, not the original, is what gets re-translated on every read
<code>summary</code>	The short AI-generated English summary
<code>photo_path</code>	Filename of an attached photo in <code>uploads/</code> , or empty
<code>status</code>	<code>open</code> or <code>resolved</code>
<code>ward</code>	Which ward this complaint was filed for — determines which worker's queue it lands in
<code>created_at</code>	When it was filed

Notice there's no separate table for wards, languages, or anything else — a ward is just a text string that happens to match between a worker's `ward` column and a complaint's `ward` column. That's intentionally simple for a first version (see §11 for what a more structured version might look like).

Authentication & roles, step by step

What a JWT actually is

When you log in successfully, the backend hands your browser a long string — a JWT (JSON Web Token). It has three dot-separated parts: a header, a payload (your user ID, your role, and an expiry time), and a signature. The signature is what makes it trustworthy: it's created using a secret key *only the backend knows*, so the backend can always tell whether a token was genuinely issued by it or tampered with.

The browser stores this token (in `localStorage`) and attaches it to every future request as an `Authorization: Bearer <token>` header. The backend never has to "remember" who's logged in between requests — it just re-verifies the token's signature every single time, in `backend/deps.py`'s `get_current_user`.

NOTABLY

This project implements JWT signing itself, using only Python's standard library (`hmac`, `hashlib`, `base64`) rather than a third-party JWT package — reasonable here because the app only ever needs to verify tokens it issued to itself, with one shared secret, nothing more exotic.

Sign-up vs. login vs. role checks

- 1 **Sign-up** (`POST /auth/signup`) always creates a `citizen` account. There is no "role" field a signer-upper can set — it's hardcoded.
- 2 **Login** (`POST /auth/login`) works identically for all three roles — same endpoint, same phone+password form. The response includes the role, and the frontend routes you to the matching dashboard.
- 3 Every protected backend route depends on either `get_current_user` (just "is someone logged in") or `require_role("worker", "admin")` (that, plus "and are they

one of these roles") — see `backend/deps.py` .

4

On the frontend, `<ProtectedRoute allowedRoles={[ ... ]}>` does the same job for pages: it won't render a page at all unless the logged-in user's role is allowed, redirecting to login or to that user's correct home page otherwise.

Both checks exist because either one, alone, isn't enough: the frontend check exists purely for a good user experience (no flash of a page you shouldn't see); the backend check is the one that actually matters for security, since a determined user could call the API directly, bypassing the frontend entirely.

Three real journeys through the app

A. A citizen reports a garbage problem, by voice

- 1 Opens the app, picks **Marathi** on the language gate, signs up with name/phone/password.
- 2 On the citizen dashboard, taps  **Speak**, records a few seconds describing the problem, picks their ward from the dropdown (only wards with an actual worker assigned are shown), and hits Submit.
- 3 The browser packages the recording as a file and `POST`s it to `/complaints` along with the language code and ward.
- 4 The backend's `ComplaintAgent` runs the recording through Sarvam's speech-to-text, cleans up the resulting Marathi text, translates it to English, and asks the chat model for a short summary — then saves one row in `complaints`.
- 5 The citizen immediately sees "Complaint submitted successfully" and the new complaint appears at the top of their own list, marked `OPEN`.

B. A worker resolves it

- 1 Logs in with the phone/password their admin gave them — lands straight on their own ward's queue.
- 2 Sees the new complaint, translated into *their* preferred language (say, English), with the short summary above the full text.
- 3 After fixing it in the real world, taps **Mark Resolved**.

- 4 The frontend sends `PATCH /complaints/{id}` with `{"status": "resolved"}`. The backend checks the complaint's ward matches the worker's own ward before allowing it.

C. An admin adds a new worker

- 1 Logs into the admin dashboard, sees every existing worker and their open/resolved counts across all wards.
 - 2 Clicks **+ Add worker**, fills in a name, phone, temporary password, and the ward name exactly as it should appear to citizens.
 - 3 The backend creates a `worker` -role account tied to that ward. The admin shares the phone/password with the real person out-of-band (there's no invite email system yet).
 - 4 That ward now shows up automatically in every citizen's ward dropdown — nothing else needs to be configured for complaints to start reaching this worker.
-

What actually breaks, and where

A separate investigation (`docs/ai_pipeline_limits.md`) tested the AI pipeline against the real Sarvam API rather than guessing. Two findings matter enough to know before demoing this:

30s HARD CAP ON ONE VOICE RECORDING	~75% NORMALIZE FAILURE RATE AT 500 WORDS	6.7–36s SUMMARY LATENCY RANGE, TYPICAL COMPLAINT	3 ROLES, EXACTLY — NO MORE, NO FEWER
--	---	--	---

REAL LIMIT — VERIFIED LIVE

Sarvam's speech-to-text endpoint (the one this app uses) hard-rejects any recording over **30 seconds** with an explicit error. Nothing in the app currently warns a citizen while recording, or explains this if it happens — a long-winded complaint just fails with a generic error message.

REAL LIMIT — VERIFIED LIVE

Both the Normalize and Summarize steps use a "reasoning" AI model whose response time and reliability vary a lot per call, not just with input length — a typical, short complaint's summary took anywhere from **6.7 to 36 seconds** across repeated identical tests. On this branch specifically, if Summarize fails, the whole complaint submission currently fails too (a fix making this fall back gracefully, like Normalize already does, exists on a separate branch but hadn't reached this one at the time of writing).

What's explicitly not built yet

From `future_work.md` — the project is honest about scope. Not yet built: PostgreSQL (still SQLite), photo analysis, automatic category/priority detection, live status updates without refreshing, maps, an accept/reject workflow for workers, structured ward-to-address

geography (a ward today is just a matching text string), password reset / OTP verification, and rate limiting on login.

Running the whole thing locally

1. Backend

- 1 Install Python dependencies: `pip install -r requirements.txt`
- 2 Copy `.env.example` to `.env` and fill in a real `SARVAM_API_KEY` (get one at sarvam.ai) and a `JWT_SECRET_KEY` (any long random string).
- 3 Start the server: `uvicorn backend.main:app --reload` — API docs auto-generate at `http://localhost:8000/docs`.
- 4 Seed the first admin account: `python scripts/seed_admin.py --phone 9999999999 -password "change-me" --name "Your Name"`

2. Frontend

- 1 `cd frontend-react && npm install`
- 2 `npm run dev` — opens at `http://localhost:5173`

3. Try the three roles

Sign up as a citizen through the app itself. Log in as the admin you just seeded, add a worker for a ward, then log in as that worker in a different browser (or incognito window) to see both sides at once.

How this is tested

Two separate test suites, for two separate layers:

- **Backend (pytest):** `pytest tests/ -v`. All external AI calls are mocked, so the whole suite runs with no API key configured and no real network calls. One exception — `tests/test_ai_limits.py` has two opt-in tests that hit the live Sarvam API, only when explicitly run with `RUN_LIVE_AI_TESTS=1`.
- **Frontend (Playwright):** `npm run test:e2e`, from inside `frontend-react/`. Drives a real browser through language selection, signup/login, protected-route redirects, the admin→worker flow, and theme/voice-recording UI.